École Polytechnique Fédérale de Lausanne

Faster Stream Joins through Parallelization and Indexing

by Yuchen Ouyang

Master Project Report

Anastasia Ailamaki
Thesis Advisor

Liang Liang, Eleni Zapridou
Thesis Supervisor

EPFL IC IINFCOM DIAS
BC 242 (Bâtiment BC)
Station 14
CH-1015 Lausanne

June 22, 2025

Abstract

Stream joins are vital for modern applications but challenging to accelerate for large windows while maintaining correctness. This research investigates enhancing stream join performance via advanced parallelization (comparing Broadcast and Handshake Joins, optimizing Handshake for deadlocks/load imbalance) and novel indexing (traditional B+ Tree vs. learned Alex). Anticipated findings include optimized Handshake's superior scalability, significant throughput gains from its enhancements, and a nuanced view of learned indexes. While Alex shows promise (e.g., 20% average gain over B+ Tree on uniform data), its performance is data-dependent and can cause worker processing imbalance. The work aims to provide an evaluation framework, a faster Handshake Join, and the first systematic assessment of learned indexes in parallel stream joins, offering insights into their utility and limitations.

Chapter 1

Introduction

The increasing availability of real-time data from sources like social media, IoT devices, and financial transactions has made stream processing a key part of modern application design. A core function in stream processing is the stream join, which combines data points (tuples) from two or more streams based on specific rules (join predicates) to find related information. Unlike typical database joins that work on fixed, unchanging data, stream joins handle data that flows continuously, can be endless, and often arrives very quickly.

To manage this endless flow of data, stream joins usually operate within a "window." This window defines a limited section of the streams—for example, data from the last minute or the most recent million entries—on which the join is performed. When new data arrives in one stream, it is checked against the data in the other stream's window for matches. After this check, the new data is added to its own stream's window, and the oldest data that falls outside the window's limits is removed[5]. However, today's applications often need very large windows (e.g., a million data points or more) for useful analysis. This makes simple, single-threaded join methods too slow. Therefore, we need ways to speed up processing while ensuring the join results are always correct, meaning each result is produced exactly once.

Parallelization and indexing are two main approaches to accelerate stream joins. Parallelization involves spreading the join workload across multiple worker threads, with each thread handling a portion of the data window. While promising, parallelization brings challenges like higher communication costs between workers, extra synchronization effort, and the difficulty of ensuring that each result is produced exactly once without missing or duplicating any. The way data is distributed and how workers coordinate—known as the join diagram—is vital for reducing these problems. At the same time, effective indexing methods within each worker can speed up local join processing by making probe operations within the data sub-windows much faster.

This research project focuses on a systematic investigation into these acceleration techniques. The core contributions include:

1. Comparative Analysis of Parallel Join Diagrams: This research conducts a performance comparison of distinct parallel join strategies, specifically Broadcast Join and Handshake Join, evaluated under diverse operational conditions.

2. Optimized Handshake Join Implementation: We significantly enhance the Handshake Join by overcoming its initial constraints, including input rate limitations, and by effectively mitigating subsequent issues such as deadlocks and load imbalance.

3. Evaluation of Learned Indexes in Parallel Stream Joins: This work presents an initial evaluation of learned index structures (specifically Alex) for their application in local join processing within parallel stream join workers.

The communication overhead associated with parallel join diagrams is a critical factor. For instance, in a Broadcast Join, the master thread must duplicate and send every tuple from one stream (e.g., R-tuples) to all workers, a process that becomes a significant bottleneck as the number of workers increases, leading to worker idle time. This project seeks to quantify these costs and identify more scalable alternatives. By addressing these aspects, this work aims to provide a clearer understanding of how to design and implement high-performance, scalable, and semantically correct stream join systems.

Chapter 2

Background

The request for efficient stream join processing has driven the creation of various techniques, mainly focusing on parallel execution and optimized data access. This section will review existing methods relevant to this project, including common parallel join diagrams and indexing approaches, and explain how this current work fits in.

2.1 Parallel Stream Join Diagrams

Our study will compare two key parallel stream join diagrams: Broadcast Join and Handshake Join.

2.1.1 Broadcast Join

In a Broadcast Join architecture (Figure 2.1), a single master thread is responsible for receiving incoming tuples from both streams. Tuples from one stream (say, R) are typically broadcast to all worker threads. Tuples from the other stream (S) are often unicast to specific workers, or also handled by the master for dispatch. Each worker maintains a local sub-window and performs join operations on the data it receives. While conceptually simple, the broadcast operation for the R-stream can become a significant communication bottleneck, as the master must replicate and transmit each R-tuple to every worker. This overhead limits the scalability of the Broadcast Join, especially as the number of workers increases.

2.1.2 Handshake Join

The Handshake Join, with notable contributions from Teubner et al. [6], offers an alternative parallel architecture. In this model, worker threads are typically arranged in a pipeline. The two input streams, R and S, flow in opposite directions through this pipeline of workers (Figure 2.2). For instance, R-tuples might flow from worker W_1 to W_N, while S-tuples flow from W_N to W_1. Each worker performs joins between the R-tuples it receives from its "upstream" R-neighbor (or the input source) and the S-tuples in its local window, and vice-versa for S-tuples. Tuples are passed from one worker to the next, often based on conditions related to buffer occupancy or tuple counts

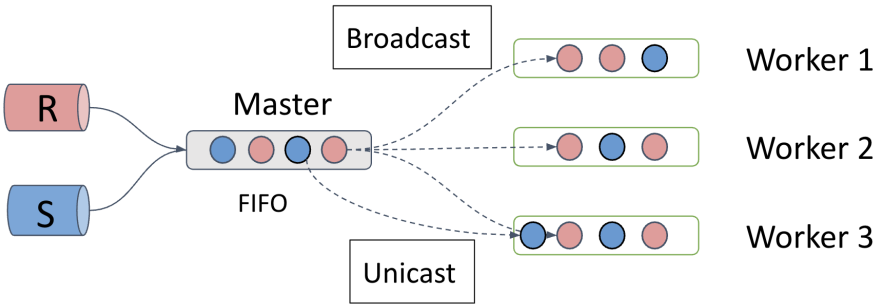

Figure 2.1: Broadcast Join Diagram

to maintain a balanced flow. The original implementations of Handshake Join, as described in foundational papers, were found to be slow in practice. This was often due to explicit input rate limitations imposed to prevent internal system issues like deadlocks or severe load imbalance, which would otherwise arise if streams flowed too quickly through the fixed-size inter-worker channels. This project's optimization efforts directly target these limitations.

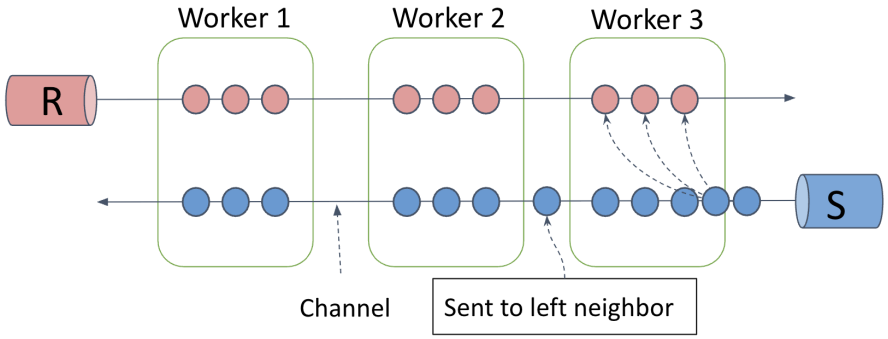

Figure 2.2: Handshake Join Diagram

2.2 Indexing for Local Join

Within each worker processing its sub-window, efficient data structures are necessary to perform the join's probe phase quickly.

2.2.1 B+ Tree

The B+ Tree is a well-established, balanced tree data structure widely used in database systems for efficient searching, insertion, deletion, and sequential access. In the context of stream joins, a B+ Tree can be maintained on the join attribute of tuples within each worker's local window. When a new tuple arrives, it probes the B+ Tree of the opposing stream's window to find matching tuples

within the specified join range. B+ Trees offer logarithmic time complexity for these operations and are robust across various data distributions, making them a standard choice for indexing.

2.2.2 Learned Index

Recently, learned index structures have emerged as a promising alternative to traditional indexes like B+ Trees [4]. The core idea, exemplified by structures like Alex [1] and PGM-Index [2], is to leverage machine learning models to learn the underlying data distribution. Instead of relying on rigid comparison-based tree traversal, a learned index (e.g., a recursive model index using piecewise linear functions) attempts to directly predict the position or a small search range for a given key (Figure 2.3). If the data distribution is well-behaved and can be accurately modeled (e.g., exhibiting good linearity), learned indexes can offer significant performance improvements over B+ Trees by reducing cache misses and computational overhead. However, their performance is inherently tied to the accuracy of the learned model, which can degrade if the data distribution is complex, skewed, or changes over time.

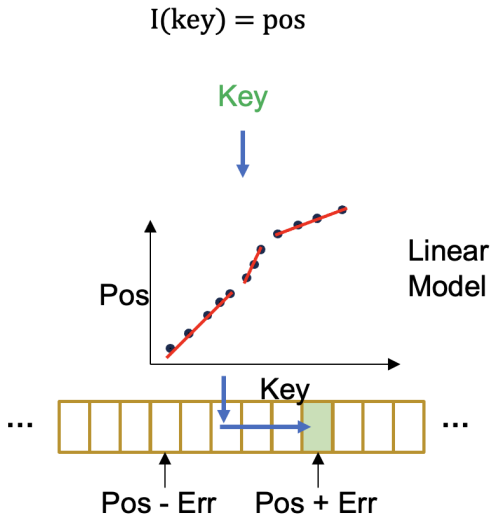

Figure 2.3: Learned Index

Chapter 3

Design

The design of the accelerated stream join system encompasses two primary dimensions: parallelization strategies to distribute workload and indexing techniques to speed up local join computations within each parallel unit. This section details the architecture of the join diagrams investigated, the optimizations applied, and the indexing mechanisms employed.

3.1 Parallelization Strategies

The core idea of parallelization is to divide the large join window into smaller sub-windows, each managed by a worker thread, to process incoming tuples in parallel.

3.1.1 Broadcast Join

The Broadcast Join serves as a baseline parallel architecture. It features a single master thread and multiple worker threads (Figure 2.1).

- **Tuple Dispatch**: The master receives tuples from both input streams, R and S. It then broadcasts all tuples from one stream (e.g., R-tuples) to every worker thread. For the other stream (S-tuples), the master may unicast them to a specific worker or handle their distribution in a round-robin fashion. Tuples are dispatched in their arrival order to maintain stream semantics.
- **Worker Operation**: Each worker maintains its local sub-window for both streams and performs range searches for incoming tuples against its local data. The primary tuple transmission strategies involved are broadcast (one-to-many) for the R-stream and unicast (one-to-one, from master to a worker) for the S-stream tuples after initial reception by the master.

3.1.2 Optimized Handshake Join

The Handshake Join architecture, where streams R and S flow in opposite directions through a pipeline of workers (Figure 2.2), was significantly re-engineered to overcome the performance

limitations of its original conceptualization. The original design often restricted input rates to avoid system instability [6]. The optimizations focused on removing these explicit rate limits and robustly handling the resulting complexities.

We aimed to eliminate any artificial constraints on the input stream rates, allowing the system to process tuples as fast as possible. This immediately highlighted the need for more sophisticated flow control and synchronization.

Deadlock Resolution using Pending Queues

With unrestricted tuple flow, the fixed-size channels connecting workers could quickly become full. If, for example, worker W_1 wants to send an R-tuple to W_2 (whose R-channel buffer is full) and simultaneously W_2 wants to send an S-tuple to W_1 (whose S-channel buffer is also full), a deadlock occurs as both workers block, waiting for the other to consume from the channel.

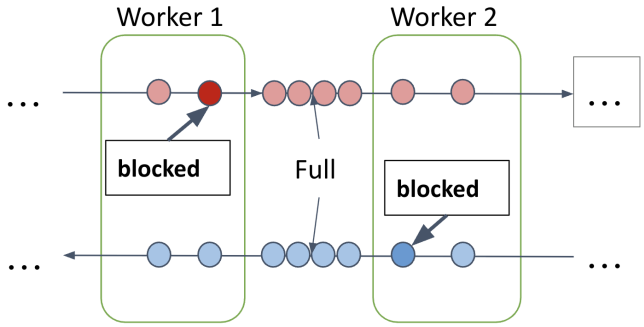

Figure 3.1: Deadlock under unlimited input rate

To resolve this, each inter-worker channel is associated with a pending queue. When a worker intends to send a tuple, it first appends the tuple to this pending queue. A separate mechanism continuously polls tuples from the head of the pending queue and flushes them to the actual channel only if the channel has empty slots.

Load Balancing Mechanisms

A critical requirement is to guarantee that each valid join pair is reported exactly once. In the Handshake Join, an acknowledgment (ACK) mechanism is employed for passing S-tuples between workers[6]. When an S-tuple is passed from worker W_i to W_{i-1}, W_i retains a copy until W_{i-1} acknowledges its processing (or further passage). This ensures that S-tuples are not lost during transit or due to worker failures before being fully processed against the relevant R-tuples.

This ACK mechanism, however, has a side effect: S-tuples flow more slowly through the pipeline compared to R-tuples (which do not use ACKs in this manner). This slower flow makes S-tuples more likely to accumulate, contributing to the observed load imbalance where S-indexes tend to be larger, especially at earlier stages of their flow.

Therefore, removing input rate limits also led to significant load imbalance, particularly an accumulation of tuples at the first worker in the pipeline for stream S. This worker would become a bottleneck, starving downstream workers and degrading overall performance.

To address this, a two-part load balancing algorithm was implemented as follows (Figure 3.2):

- **Sliding Window** (TCP-like Flow Control): This mechanism monitors the number of tuples being popped at the "other end" of the pipeline (for a given stream direction). For instance, the routine sending S-tuples monitors tuples popped by the last worker in the S-flow direction. This helps prevent overwhelming the entire system with an excessive number of in-flight tuples, keeping the total number of tuples roughly equivalent to the overall window size. This mechanism also prevents unbounded accumulation in pending queues, ensuring they do not lead to out-of-memory errors.
- **Capacity Limit at First Worker**: Since the first worker is most susceptible to accumulation, an explicit capacity limit is enforced on its local sub-window size. If this worker's buffer for a stream (e.g., S-tuples) exceeds a threshold (approximately the total window size divided by the number of workers), it prioritizes transferring its oldest tuples to the next worker in the pipeline. This helps distribute the load more evenly across all workers.

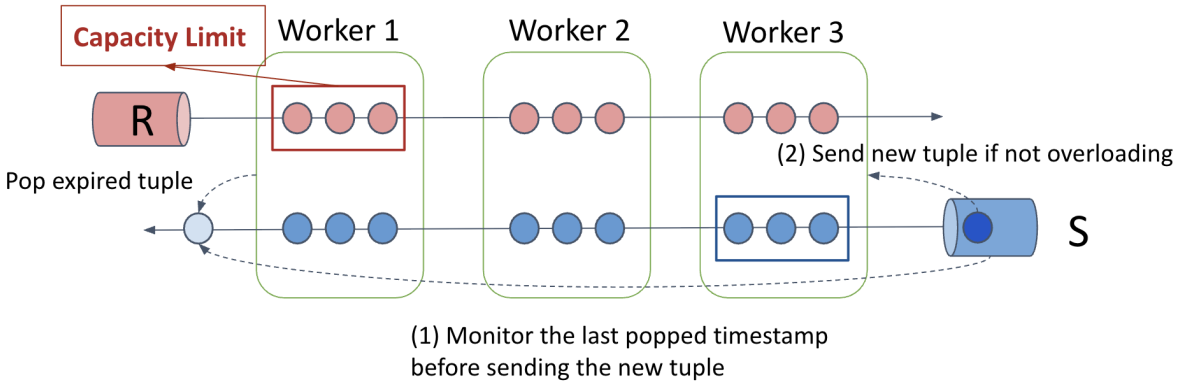

Figure 3.2: Load Balancing Mechanism

3.2 Indexing for Local Joins

Once tuples arrive at a worker, efficient local join processing is paramount. This involves probing an index built on the tuples of the opposing stream within the worker's sub-window.

3.2.1 B+ Tree

A B+ Tree is used as a standard, robust indexing method. Each worker maintains separate B+ Trees for the R-tuples and S-tuples in its local sub-window, indexed by their join attributes. When a new

tuple arrives (e.g., an R-tuple), it probes the B+ Tree of S-tuples to find all tuples within the specified join key range.

3.2.2 Alex Learned Index

Alex is an adaptive learned index structure that aims to improve upon traditional indexes by learning the data distribution.

- **Architecture**: Alex typically uses a hierarchy of models. For this project, it is conceptualized as mapping keys to positions using a piecewise linear function. The model $I(key) = pos$ (Figure 2.3) predicts an approximate position for a key, and search is then localized to a small range $[PosErr, Pos + Err]$ [1].
- **Objective**: By exploiting knowledge of the data distribution, Alex seeks to achieve faster lookups, insertions, and deletions compared to data-oblivious structures like B+ Trees, especially for favorable distributions.

Chapter 4

Experiments

4.1 Experimental Setup

- **Hardware**: All experiments were performed on a machine equipped with an Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz, featuring 2×14-core Broadwell processors with multithreading enabled.
- **Software**: The system was developed in C++17 and compiled using g++ 9.4.0, running on Ubuntu 20.04.2.
- **Default Parameters**: Unless specified otherwise, experiments involved a 2-way stream join with a window size of 1 million tuples. The data for keys was drawn from a uniform distribution, and the join predicate involved a range search with Diff/Key Range = 0.2% of the total key space.
- **Memory Management**: All join operations and data structures were maintained in DRAM to isolate processing performance from disk I/O latencies.

4.2 Data Distributions

- **Uniform**: default
- **Books Dataset**: A real-world SOSD dataset [3] characterized by good key linearity (Figure 4.1, left). This means its cumulative distribution function (CDF) is relatively smooth and can be well approximated by piecewise linear functions, a favorable condition for learned indexes like Alex.
- **OSM (OpenStreetMap) Cellids Dataset**: Another real-world SOSD dataset, but one that exhibits poor key linearity, with a more irregular and stepped CDF (Figure 4.1, right). This dataset was used to test the robustness of learned indexes under less ideal conditions and to investigate workload imbalance issues.

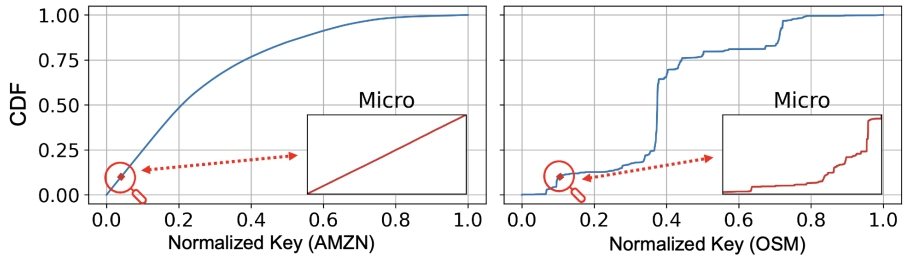

Figure 4.1: SOSD Dataset Distributions

4.3 Experimental Results and Analysis

4.3.1 Communication Cost Comparison

To isolate the inherent communication overhead of the join diagrams, an experiment was conducted where local tuple processing (join computation, index updates) at the workers was disabled. Workers only received for broadcast join or only passed tuples from its neighbor to the next neighbor for handshake join.

As shown in Figure 4.2, the throughput of the Broadcast Join drops rapidly as the number of workers increases. This is attributed to the master becoming a bottleneck as it broadcasts R-tuples to all workers. In contrast, the optimized Handshake Join (without local processing) maintained a very high throughput, around 1 million tuples/second, demonstrating the benefits of its pipelined parallelism. This high throughput represents an intrinsic performance ceiling for the Handshake diagram, limited only by inter-worker communication capabilities. The original Handshake Join implementation, constrained by explicit input rate limits, exhibited drastically lower throughput (around 4k tuples/s).

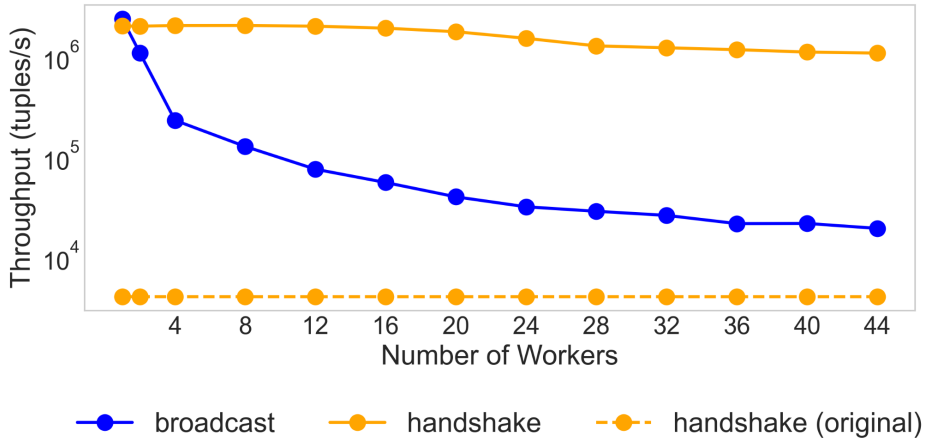

Figure 4.2: Communication Cost of Join Diagrams

It quantifies the scalability limits of Broadcast Join due to communication bottlenecks and underscores the high throughput potential of the Handshake architecture when not explicitly constrained. The poor performance of the original Handshake implementation provides strong motivation for the optimization efforts undertaken in this project.

4.3.2 Load Balancing for Handshake Join

The effectiveness of the load balancing algorithm (sliding window and first-worker capacity limit) for the optimized Handshake Join was evaluated.

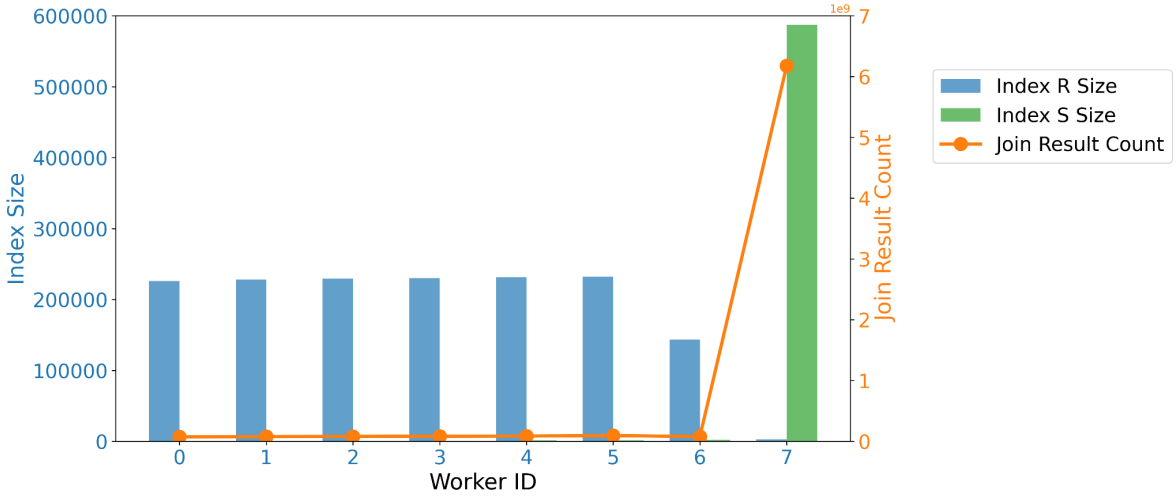

Figure 4.3: Load Imbalance without Input Rate Control

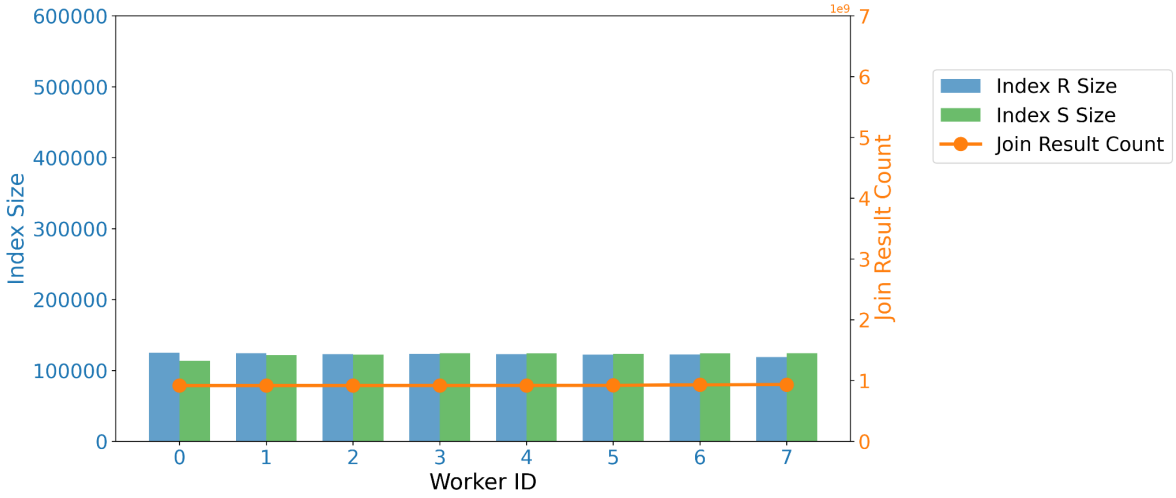

Figure 4.4: Load Balance with Input Rate Control and Capacity Limit

Without load balancing, S-tuples tended to accumulate heavily at the first worker they encountered (Figure 4.3), creating a severe bottleneck. With the load balancing algorithm deployed, the workload (measured by local index sizes) was much more evenly distributed across workers (Figure 4.4). This resulted in a substantial throughput improvement, approximately a 5× increase from around 19k tuples/s (imbalanced) to 94k tuples/s (balanced) in a representative experiment. This demonstrates the critical role of effective load balancing in realizing the performance potential of the parallel architecture.

4.3.3 Impact of Indexing on Parallel Join Performance

This set of experiments measured the performance gains from using B+ Tree indexing for local joins within parallel workers, compared to a naive list-based iteration (linear scan) over the tuples in the sub-window.

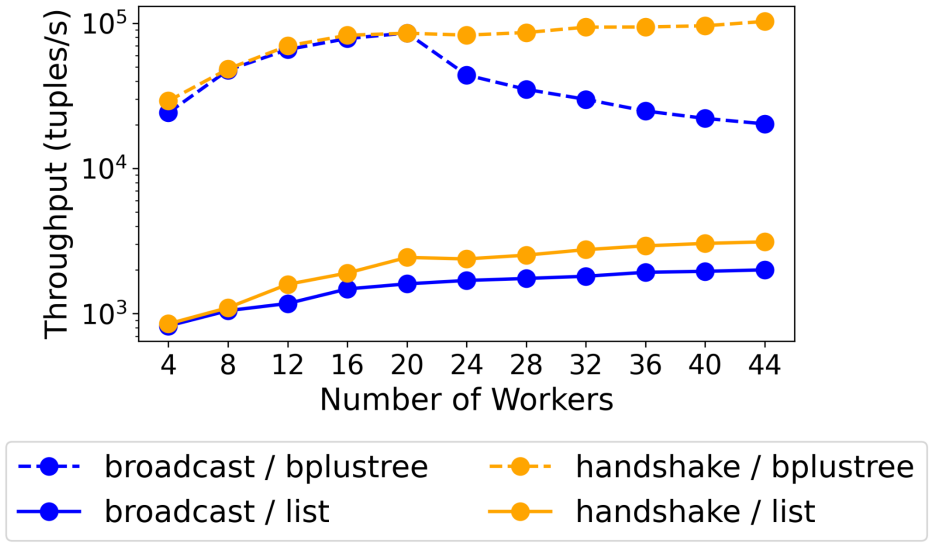

Figure 4.5: Throughput with Different Join Diagrams and Indexing Combinations

We found from Figure 4.5 that:

- Using B+ Tree indexing significantly accelerated local stream processing for both Broadcast and Handshake join diagrams.
- The Broadcast Join with B+ Tree showed initial performance improvement with more workers due to split workloads and parallelism, but its throughput eventually declined as the broadcast communication cost became the dominant bottleneck.
- The Handshake Join with B+ Tree indexing demonstrated excellent scalability, with throughput continuously growing with the number of workers, achieving approximately 100k tuples/s with a moderate number of workers and maintaining strong performance as worker count

increased. This configuration ("Handshake / B+ Tree") was the most performant among these initial comparisons.

A critical observation from these results is the "performance gap": the throughput of Handshake Join with B+ Tree indexing (~100k tuples/s) is still considerably lower than the communication-bound throughput of Handshake Join (the "upper bound" of ~1M tuples/s). This gap highlights that local join processing—including index searches, insertions, and deletions within the B+ Tree—constitutes a significant portion of the overall execution time and remains a key area for further optimization. This motivates the exploration of potentially faster local methods, such as learned indexes.

4.3.4 Comparative Performance of Alex vs. B+ Tree

The performance of the Alex learned index was compared against the B+ Tree under various data distributions. The worker size was fixed at 8, window size was 100k tuples, and the other parameters were default configurations.

Dataset	**Index Type**	**Throughput (tuples/s)**	**Alex Gain/Loss vs. B^+ Tree (%)**
Uniform	B^+ Tree	207300	
Uniform	Alex	248600	+20.0%
Books	B^+ Tree	210800	
Books	Alex	285300	+35.3%
OSM	B^+ Tree	247300	
OSM	Alex	220000	-11.1%

Table 4.1: Comparison of Throughput and Gain/Loss for Different Index Types

Given the Table 4.1, we found out that:

- In experiments with Handshake Join on uniformly distributed data, Alex achieved an average throughput of 248.6k tuples/s, compared to 207.3k tuples/s for B+ Tree. This represents an average performance gain of approximately 20% for Alex.
- On the Books dataset (good linearity), Alex significantly outperformed B+ Tree by about 35.3%.
- Conversely, on the OSM dataset (poor linearity), B+ Tree performed better than Alex.

These results (Table 4.1) are crucial for understanding the "double-edged sword" nature of learned indexes. They show the strong potential of learned indexes when the data matches what the model expects. However, they also reveal a major weakness: their performance drops significantly when the data distribution is not ideal.

4.3.5 Workload Imbalance with Learned Indexes

Given Alex's sensitivity to data distribution, further experiments investigated whether it could introduce processing time imbalance across workers, even if data sizes (number of tuples) were

balanced in 3.1.2. Handshake join workers was set to 8 on the osm dataset (known to be challenging for Alex).

We found that Alex led to more varied local processing times among workers compared to B+ Tree, even when their average times were similar. For instance, on the OSM dataset, Alex had a standard deviation of 549.36 ms, while B+ Tree had 397.17 ms (Table 4.2). This uneven workload means the slowest worker limits the overall system speed, creating a bottleneck and reducing the advantages of using parallel processing.

Index Type	**Avg. Processing Time (ms)**	**Standard Deviation of Processing Time (ms)**
B^+ Tree	1385.90	397.17
Alex	1445.22	549.36

Table 4.2: Processing Times for Different Index Types

Table 4.2 highlights a critical practical challenge for deploying learned indexes in parallel systems: ensuring processing time balance, not just data size balance. This leads to the open research question of how to transition from size-based load balancing to more sophisticated processing time-aware balancing.

Chapter 5

Future Work

Our experiments and design work have highlighted some current limitations and opened doors for future research. We still face key challenges in building super-efficient and reliable stream join systems:

- **Persistent Local Processing Gap**: Even with the major improvements to the Handshake Join using B+ Tree indexing, which achieved about 100k tuples per second, there's still a big difference when we compare it to its theoretical maximum if only communication overhead existed (around 1 million tuples per second). This tells us that local join processing—things like searching and updating indexes, and managing data within each worker—still takes up a lot of the processing time. This area is a prime target for more optimization.
- **Workload Imbalance with Learned Indexes**: Learned indexes like Alex are very sensitive to how data is distributed locally, which can cause big differences in how long each parallel worker takes to process data. Even if we balance the amount of data each worker gets, this uneven processing time can make the slowest worker a bottleneck, dragging down the entire system's speed. Figuring out how to move beyond just balancing data size to effectively balancing processing time is still an unsolved problem.

To address the identified limitations and advance the state of stream join performance, focused research in several key areas is essential:

- **Co-design of Join Diagrams and Learned Local Join Methods**: A primary direction for future work involves the holistic co-design of parallel join strategies and adaptive local join methods that leverage learned approaches. This entails a tighter integration where the selection of a join diagram could inform the type or configuration of learned indexes utilized locally, and vice-versa. For instance, a join diagram might intelligently route data to workers in a manner that optimizes data characteristics (e.g., linearity) for the local learned indexes. Such co-design aims to specifically mitigate processing time imbalance and fully harness the potential of learned structures within a parallel environment.

- **Batching Tuples for Communication Reduction**: Further investigation into batching tuples prior to their transmission between workers, or from a master node (in the context of Broadcast Join), is warranted. While acknowledging the potential trade-off with batch construction latency, particularly for low-rate streams, batching holds promise for reducing per-tuple communication overhead and enhancing network utilization for high-volume streams.

- **Scalability to Larger-than-Memory Workloads**: For scenarios where window sizes exceed available DRAM capacity, research efforts should extend to:

  Disk-Based Storage: Developing efficient disk-spilling techniques and integrating robust buffer pool management strategies will be necessary to handle data that cannot reside entirely in memory.

  Distributed Computing: If the aggregate workload (tuple arrival rate times join complexity) becomes excessively high even for a multi-core machine, sharding the streams or the window state across multiple distributed nodes could be explored. The challenge here would be to maintain high performance by keeping active data in memory at each node while managing inter-node communication and consistency.

Chapter 6

Conclusion

This research project aimed to address the critical challenge of accelerating stream join operations, particularly given the growing size of operational windows and the high velocity of data streams. We successfully developed a comprehensive framework for systematically analyzing and comparing various parallelization strategies (such as Broadcast Join and Handshake Join) and local indexing techniques (including list scans, B+ Trees, and the learned index Alex).

A significant contribution of this work was the substantial enhancement of the Handshake Join architecture. We removed its original input rate limitations and systematically resolved the resulting challenges of deadlocks, by implementing pending queues, and load imbalance, through a novel sliding window mechanism and first-worker capacity limits. These improvements led to a considerable increase in its throughput compared to previous designs.

Furthermore, this project conducted one of the initial evaluations of learned indexes within the demanding context of parallel stream join scenarios. This provided empirical evidence of their potential benefits under favorable data conditions. However, it also highlighted their practical limitations and the new challenges they introduce, such as dependency on data distribution and imbalances in processing time.

Bibliography

[1] Jialin Ding, Umar Farooq Minhas, Hantian Zhang, Yinan Li, Chi Wang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, and David B. Lomet. "ALEX: An Updatable Adaptive Learned Index". In: *CoRR* abs/1905.08898 (2019). arXiv: 1905.08898. URL: http://arxiv.org/abs/1905.08898.

[2] Paolo Ferragina and Giorgio Vinciguerra. "The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds". In: *Proc. VLDB Endow.* 13.8 (Apr. 2020), pp. 1162–1175. ISSN: 2150-8097. DOI: 10.14778/3389133.3389135. URL: https://doi.org/10.14778/3389133.3389135.

[3] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. *SOSD: A Benchmark for Learned Indexes*. 2019. arXiv: 1911.13014 [cs.DB]. URL: https://arxiv.org/abs/1911.13014.

[4] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. *The Case for Learned Index Structures*. 2018. arXiv: 1712.01208 [cs.DB]. URL: https://arxiv.org/abs/1712.01208.

[5] Amirhesam Shahvarani and Hans-Arno Jacobsen. "Parallel Index-based Stream Join on a Multicore CPU". In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD '20. Portland, OR, USA: Association for Computing Machinery, 2020, pp. 2523–2537. ISBN: 9781450367356. DOI: 10.1145/3318464.3380576. URL: https://doi.org/10.1145/3318464.3380576.

[6] Jens Teubner and Rene Mueller. "How soccer players would do stream joins". In: *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*. SIGMOD '11. Athens, Greece: Association for Computing Machinery, 2011, pp. 625–636. ISBN: 9781450306614. DOI: 10.1145/1989323.1989389. URL: https://doi.org/10.1145/1989323.1989389.